

Suivi des contributions & tâches – SecurAI / Attaque deepfake

Membre	Contributions réalisées (déjà)
Nadahe	• Architecture de base du projet • Modèle YOLOv8-face intégré • Interface web (HTML/CSS/JS) • Backend Flask (routes, gestion du flux vidéo) • Entraînement du premier modèle de reconnaissance et classification nommé <code>best_model.pt</code>
Traoré (toi)	• Implémentation de l'attaque FGSM (client & serveur) • Superposition du modèle FaceNet sur le crop YOLO • Entraînement du défenseur adversarial (defender) • Feature "mode statique" pour injection d'attaque sur image • Recherche d'une solution cloud GPU (analyse de coût)
François	-
Yassin	-

Tâches à réaliser (globales)

1. **Lunettes en live** – Générer un patch de lunettes via FGSM en temps réel et l'appliquer dynamiquement sur le visage dans le flux vidéo, exposé via `/api/generate_glasses_attack`.
2. **Documentation** – Rédiger un README complet, créer un diagramme d'architecture et un guide d'installation.
3. **Tests unitaires** – Écrire des tests `pytest` pour le détecteur de visage, le reconnaître et l'endpoint d'enrôlement.
4. **Gestion du GPU** – Script `toggle_gpu.py` pour basculer entre CPU et GPU (T4) et redémarrer le serveur.
5. **Anti-spoofing** – Vérifier la provenance d'une image via un JWT signé par le client mobile.
6. **Renforcement du défenseur** – Ajouter des transformations aléatoires (flou gaussien, compression JPEG) avant l'inférence et évaluer la robustesse.

7. **Dashboard d'administration** – Page `admin.html` affichant FPS, latence HF, mode actuel et logs, rafraîchie par polling.
8. **Export PDF** – Bouton générant un rapport PDF des métriques (FPS, latence, score FGSM) et des logs d'anomalies.
9. **Déploiement automatisé** – Script `deploy.sh` qui pousse le dépôt, crée/actualise le Space et active le GPU avec l'option `--gpu`.

Vous pouvez maintenant visualiser le fichier à jour. N'hésitez pas à indiquer quelle tâche vous souhaitez développer en priorité ou à demander des précisions supplémentaires